Overlapping communities as connected components of a commonality hypergraph

Dmitri Deinega,[1] Igor Kanovsky,[2] and Baruch Barzel[1]

[1] *Department of Mathematics, Bar-Ilan University, Ramat Gan 5290002, Israel*
[2] *Max Stern Yezreel Valley College, Emek Yezreel, Israel*

Nodes that share most of their neighbours tend to share a community. On this premise, three nodes, one adjacent to the other two, whose three pairs all reach a commonality threshold form an elementary community, and communities are the connected components of the resulting 3-uniform hypergraph under shared pairs. The construction, HyperCommon, is deterministic, local and nested in its threshold, and no pair of nodes binds two communities. Its cost is linear in the number of wedges, which an exact pruning of hub wedges keeps below $12\,m\ln m$ on thirty-two real networks with m edges. We derive the exact window of thresholds that recovers planted overlapping ring lattices. Compared with fourteen other methods, every parameter chosen without labels, HyperCommon is the most accurate in fifteen of twenty-eight ring-lattice settings and level in six, most clearly where communities of unequal size and density overlap. It trails on all eight families of overlapping LFR graphs, and on the rings once a tenth (with labels) or three tenths (without) of their edges are rewired internally: it needs communities whose members share many neighbours. On seventeen real networks of up to 1.1×10^6 nodes, the threshold chosen without labels keeps a median 94% of the best threshold's F1, but on most of them another method is more accurate.

I. INTRODUCTION

Most real networks are organised into groups of nodes that are more densely connected to one another than to the rest of the network [1, 2], and in social, biological and information networks these groups overlap. Methods for overlapping communities [3, 4] differ less in how they search than in what they take a community to be: a group compared with a global null model [5, 6], with a random walker's trajectories [7], with a generative model [8, 9], with its significance against a configuration-model null [10], or with a consensus among neighbours [11–13]. No definition is best on every network [14, 15]; the useful question about a method is which structure it captures, and why.

A natural definition comes from the inside: two friends who share most of their friends are, more likely than not, in the same circle. This is triadic closure [16]. Real networks are highly clustered [17], and neighbourhoods overlap more between members of a group than across groups, although separating the two effects is itself an inference problem [18]. We call the overlap of two closed neighbourhoods the *commonality* of the pair. Pairwise neighbourhood similarity is an old tool [19–23], but a single pair is a weak and degree-confounded signal; what matters is how pairwise evidence is combined.

We combine it through triples. Three nodes of which one is adjacent to the other two, and whose three pairs all have commonality at least t, form an elementary community; two such triples belong to the same community when they share a pair. The communities are the connected components of the two-node-overlap graph of a 3-uniform hypergraph, which gives the method its name, HyperCommon. Higher-order structures are increasingly seen as natural units of network organisation [24, 25]; here the hypergraph is built from pairwise commonality rather than given. A node shared by two communities belongs to triples in both, so overlap needs no extra machinery.

The paper makes three contributions. (i) *The premise, tested*: on four networks with recorded memberships, edges inside recorded groups have higher commonality than edges between them, on the three large ones at every endpoint degree, once unlabelled nodes are no longer counted as evidence of separation (Sec. III). (ii) *A construction with exact guarantees*: HyperCommon is independent of processing order, local and nested in its threshold, every edge it admits belongs to exactly one community, and on overlapping ring lattices we derive the exact window of thresholds that recovers the planted communities (Secs. IV and V). Its cost is linear in the number of wedges, and we show why that number stays near $m\ln m$ on real networks with hubs (Sec. V C). (iii) *A map of where it works*: against fourteen other methods on ring lattices, LFR graphs and seventeen real networks (Secs. VI and VII), HyperCommon leads where communities are lattice-like and overlap with unequal sizes and densities, and trails where communities are random graphs internally, because their members share too few neighbours.

II. RELATED WORK

Neighbourhood overlap. That adjacent nodes sharing many neighbours tend to share a community underlies the edge clustering coefficient [26], the structural similarity of SCAN [27], shared-near-neighbour clustering [28] and the link between overlap and tie strength [29]; the same indices are the local predictors of link prediction, where no index is best on every network [30]. Jarvis–Patrick clustering, TECTONIC [31] and its refinement [32] threshold scored edges and take connected components; our two-test ablation is of this kind, restricted to wedges.

Clique and triad percolation. Clique percolation [33]

joins k-cliques sharing $k-1$ nodes, also on graphs thresholded by link weight [34, 35]; for $k = 3$ these are also the truss communities [36–38]. Our triangles-only ablation is 3-clique percolation on the graph thresholded at $J \geq t$ (closed-neighbourhood Jaccard). Triad percolation [39] already admits open triads, growing seeds until a stopping rule is met; HyperCommon admits a wedge only when all three of its pairs pass and needs no seeds or stopping rule.

Link communities. Ahn, Bagrow and Lehmann [40] call two edges $\{c, i\}$ and $\{c, j\}$ similar when $N[i]$ and $N[j]$ overlap, with similarity $J(i, j)$, and cluster edges by single linkage; hyperlink communities extend this to higher-order networks [41]. In our language $J(i, j)$ is the commonality of the third pair of the triple $\{i, c, j\}$, the pair that is not an edge when the wedge is open. Line and clique graphs [42, 43] frame such clusterings, and our two-node-overlap graph is the 2-line graph of the hypergraph [44].

What HyperCommon combines. It tests the two edges of a wedge, as thresholded clique percolation and SCAN do, and its non-adjacent pair, as link communities do, and joins admitted triples through any shared pair. Determinism and nesting in the threshold are shared with thresholded clique percolation and single-linkage link communities; what HyperCommon adds is the exact-recovery window (Theorem 2), which ties the threshold to the structure it recovers. To choose the threshold without labels we use the overlapping modularity of Shen *et al.* [45], which peaks near the critical point of k-clique percolation [46]; for partitions it is the Newman–Girvan modularity, whose resolution limit [47] the absolute Potts model [48] and the Constant Potts Model [49] avoid. The construction was first formulated, in an order-dependent form, in Ref. [50].

III. COMMONALITY IN EMPIRICAL NETWORKS

The premise is local: two adjacent nodes whose neighbourhoods largely coincide belong, more often than not, to a common community. We test it on Zachary's karate club [51] and on the DBLP, Amazon and YouTube networks of SNAP [52], whose recorded groups are publication venues, product categories and user groups (Supplemental Material [53], Table S1).

Unlabelled nodes and degree. The SNAP networks label only part of their nodes: 82% in DBLP, 95% in Amazon and 4.6% in YouTube. Counting pairs with an unlabelled node as pairs in different groups, as the first author's earlier analysis did [50], is the standard pitfall of learning from positive and unlabelled data [54]; on DBLP it reverses the conclusion, the area under the ROC curve (AUC) falling to 0.41. We therefore call an edge *inner* when its endpoints are labelled and share a recorded group and *outer* when they are labelled and share none, and compare these two classes only. Metadata need not coincide with structural communities [14, 55], so this tests a tendency, not a classifier. With the correction, commonality separates the classes on all four networks (AUC 0.82, 0.62, 0.60 and 0.61). Jaccard commonality is capped by the degree ratio, $J(u, v) \leq (d_{\min} + 1)/(d_{\max} + 1)$, and degree also correlates with how labels were assigned, but within strata of the larger endpoint degree the separation persists at every degree [Fig. 1(a)].

A family of functions. With $I = |N[u] \cap N[v]|$ and $U = |N[u] \cup N[v]|$ we compare eight overlap scores of link prediction [20, 21, 56]: I, Jaccard I/U, I^2/U, $I/U^{1/2}$, the cosine [57], $I/\min$ (after Ravasz *et al.* [58]), $I/\max$ and resource allocation; I^2/U and $I/U^{1/2}$ rank pairs alike, so seven differ for any threshold. Within strata of both endpoint degrees every one separates the classes and the overlap-based ones perform alike [Fig. 1(b)], since at fixed degrees each is a monotone function of I. They differ on pairs of different degrees, which a global threshold must compare: pooled over all edges, I leads on DBLP but falls below chance on YouTube, where hubs dominate. HyperCommon accepts any member; unless stated otherwise we use Jaccard, the similarity of link communities [40].

From pairs to triples. HyperCommon requires all three pairs of a wedge to pass, including the pair that is not an edge. Call a labelled triple *pure* when one recorded group contains all three nodes, and compare the *pair rule*, which tests only the edges of the triple, with the *triple rule*, which also tests the non-adjacent pair. At equal recall from 0.1 to 0.4 the triple rule is the more precise on all three SNAP networks [Fig. 1(c)]: at recall 0.2, 81% of pair-admitted triples are pure against 93% of triple-admitted ones in DBLP. Beyond recall ≈ 0.5, where most wedges are admitted, the pair rule is the more precise on DBLP (Fig. S3).

IV. THE METHOD

HyperCommon has two settings: the commonality function f, which scores a pair of nodes (closed-neighbourhood Jaccard unless stated otherwise), and the threshold t. Given a graph G:

1. For every wedge $\{i, j, \ell\}$ – three nodes, one adjacent to the other two, each counted once whether open or closed – compute f on its three pairs.

2. Admit the wedge as a hyperedge if all three values are at least t.

3. Join two admitted triples when they share two nodes, and return, for every connected component, the union of its triples' nodes.

Commonality values are memoised per pair, and components are found on the bipartite incidence graph between admitted triples and the pairs they contain, which has the same triple components as the two-node-overlap graph and is linear in the number of triples. Because

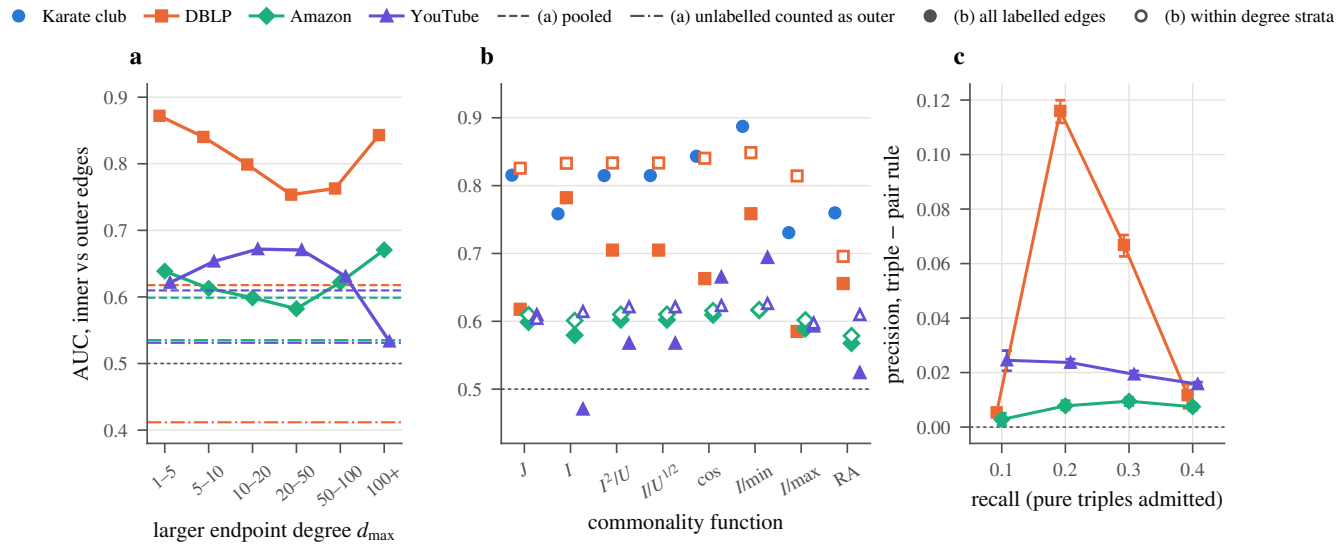


FIG. 1. Commonality in networks with recorded memberships. (a) AUC of closed-neighbourhood Jaccard for inner against outer edges within strata of $d_{\max}$ (points), pooled (dashed), and with unlabelled edges counted as outer (dash-dotted), as in Ref. [50]; the karate club is too small to stratify. (b) AUC of eight functions over all labelled edges (filled) and within strata of both endpoint degrees (open). (c) Precision of the triple minus the pair rule at equal recall, with 95% intervals from resampling the triples.

the triple scores $s(T) = \min f$ over the pairs of T do not depend on t, the covers for every threshold follow from a single enumeration, each at the cost of one component search. The threshold can be set from the graph alone, by maximising the overlapping modularity of the cover [45] (Sec. VI C). A local variant grows the community of a given seed triple by breadth-first search over shared pairs, reading only the neighbourhoods of the nodes it reaches and of the candidates it examines (Proposition 2).

Figure 2 illustrates the construction. Nodes in no admitted triple remain unassigned, and communities have at least three members: the method reports only what local evidence supports (coverage in Table S13 of the Supplemental Material [53]).

V. PROPERTIES OF THE COMMONALITY HYPERGRAPH

Throughout, $G = (V, E)$ is a finite simple undirected graph, $N[u] = \{u\} \cup N(u)$ is the closed neighbourhood of u, and

$$J(u, v) = \frac{|N[u] \cap N[v]|}{|N[u] \cup N[v]|} \tag{1}$$

is the commonality of the pair. A set $T = \{i, j, \ell\}$ of three distinct nodes is a *wedge* if one of its nodes is adjacent to the other two, and it is *admissible at threshold* t if it is a wedge and $J \geq t$ for all three of its pairs. The admissible triples $\mathcal{T}_t$ are the hyperedges of a 3-uniform hypergraph $\mathcal{H}_t = (V, \mathcal{T}_t)$. Let L_t be its two-node-overlap graph, whose vertices are the triples of $\mathcal{T}_t$, two of them adjacent when they share exactly two nodes. The communities at threshold t are the node sets of its components:

$$\mathcal{C}_t = \Big(\bigcup_{T \in K} T\Big)_{K \text{ a connected component of } L_t}. \tag{2}$$

A node belongs to every community whose component contains a triple through it, so $\mathcal{C}_t$ is a cover, and nodes in no admissible triple are left unassigned. All results except Lemma 1 and Theorem 2, which are specific to Jaccard, hold for any commonality function; proofs are in Appendix A.

A. Determinism, locality and nesting

Proposition 1 (Order independence). *$\mathcal{C}_t$ is a function of (G, t) alone: it does not depend on a processing order, a random seed, or the order in which triples are discovered.*

This is the property the original formulation [50] lacked: it deleted each edge once used, so whether a later triple could still be formed depended on which community had been grown first.

Proposition 2 (Local query). *Let $T_0 \in \mathcal{T}_t$. Breadth-first search over L_t from T_0, in which a frontier pair $\{x, y\}$ is extended by every $w \in N(x) \cup N(y)$ with $\{x, y, w\} \in \mathcal{T}_t$, visits exactly the component of L_t that contains T_0, and evaluates J only on pairs within the closed neighbourhoods of the nodes it reaches.*

Theorem 1 (Nesting). *If $t_1 \leq t_2$, then L_{t_2} is the subgraph of L_{t_1} induced on $\mathcal{T}_{t_2}$, and every community of $\mathcal{C}_{t_2}$ is contained in a community of $\mathcal{C}_{t_1}$.*

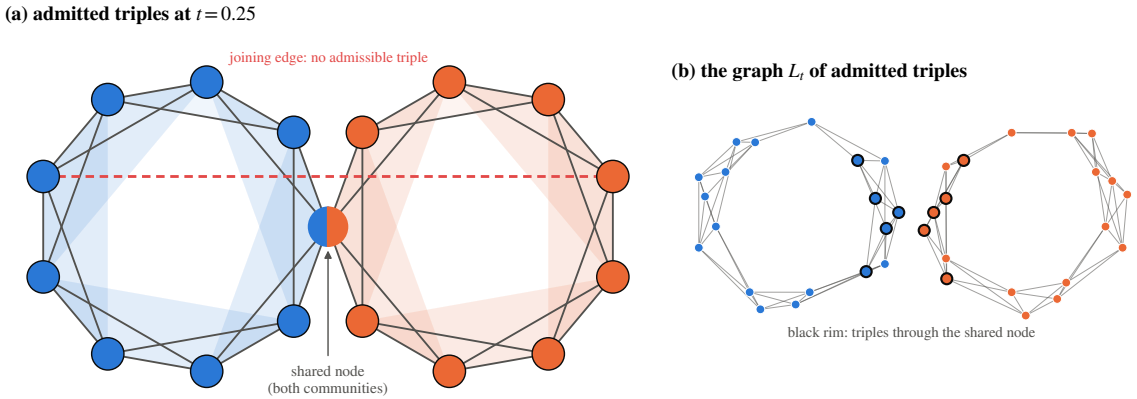


FIG. 2. The method on a toy graph, computed by the implementation. (a) Two ring lattices of nine nodes ($k = 4$) share one node, and one edge, placed by hand for the illustration between two nodes far from the shared node (it closes no triangle with it), joins them. Shaded triangles are the triples admitted at $t = 0.25$, coloured by component; the shared node is admitted in both, the joining edge in none. (b) The graph of admitted triples, linked when they share a pair (L_t), each component drawn on its own circle; its two components are the two communities. The rings have $s = 9 < 2k + 2$, outside the stated hypothesis of Theorem 2 but within the ring sizes for which its window still holds (Sec. S3).

Raising t only splits communities or removes nodes; it never merges two of them. The covers at all thresholds therefore form one hierarchy, which changes only at the finitely many values J takes. Its two ends carry no information: on a connected graph the cover at $t = 0$ is one community, and at $t = 1$ only classes of nodes with identical closed neighbourhoods remain, so the most accurate threshold is interior (Proposition S1 of the Supplemental Material).

B. Exact recovery on overlapping ring lattices

Consider r rings of sizes $s_i \geq 2k + 2$, each node joined to its $k/2$ nearest neighbours on either side ($k \geq 4$ even), and a set of *merges*: a node v of one ring is absorbed into a node u of another, which inherits v's edges and so belongs to both ground-truth communities. The merges are *isolated* if each original node takes part in at most one merge and, on each original ring, any two merge positions are more than k apart.

Lemma 1 (Commonality on the lattice)**.** *With isolated merges, let x, y be unmerged nodes at ring distance $d \leq k$, u a merged node at distance d from x along one of its rings, and z, y' neighbours of u on its two different rings. Then*

$$\begin{aligned} J(x, y) &= J(d) \equiv \frac{k + 1 - d}{k + 1 + d}, \\ J(u, x) &= J_u(d) \equiv \frac{k + 1 - d}{2k + 1 + d}, \\ J(z, y') &= \frac{1}{2k + 1}, \end{aligned} \tag{3}$$

and every other pair on distinct rings has $J = 0$.

Theorem 2 (Exact recovery window)**.** *With at least one merge, all of them isolated, $\mathcal{C}_t$ equals the ground truth if and only if*

$$\begin{aligned} &\frac{1}{2k + 1} < t \leq t^+(k), \\ &t^+(k) \equiv \min\Big\{ J_u(1),\, \max\big(J(3), J_u(2)\big) \Big\} \\ &\qquad = \min\Big\{ \frac{k}{2k + 2},\, \max\Big(\frac{k - 2}{k + 4}, \frac{k - 1}{2k + 3} \Big) \Big\}. \end{aligned} \tag{4}$$

Without merges the condition is $0 \leq t \leq J(2) = (k - 1)/(k + 3)$.

Explicitly, $t^+(k) = k/(2k+2)$ for $k \geq 8$, so the window widens towards $(0, \frac{1}{2})$ as k grows: $(1/17, 4/9]$ for $k = 8$ and $(1/33, 8/17]$ for $k = 16$. Both edges have a direct reading. The lower edge is the price of overlap: a shared node is the only bridge between its two rings, and the bridge is cut once t exceeds the commonality $1/(2k+1)$ of the rings' members across it. The upper edge is set by the shared nodes themselves: their commonality with either ring is roughly halved by their double membership, and above it they drop out. For one isolated merge between rings of degrees k_a and k_b the same argument gives

$$\begin{aligned} &\frac{1}{k_a + k_b + 1} < t \leq \min\{U(k_a, k_b), U(k_b, k_a)\}, \\ &U(x, y) = \min\Big\{ \frac{x}{x + y + 2},\, \max\Big(\frac{x - 2}{x + 4}, \frac{x - 1}{x + y + 3} \Big) \Big\}, \end{aligned} \tag{5}$$

and a family with several merge types is recovered exactly if and only if t lies in the intersection of their windows; once $k_a \geq 2k_b^2 - 2$ the windows of dense–sparse and sparse–sparse merges are disjoint, and no single threshold recovers a family with both. We verified these windows exactly, in rational arithmetic, for $k = 4, \ldots, 24$ and eleven degree pairs, together with the smaller-ring

and multi-ring cases of Sec. S3 of the Supplemental Material.

Proposition 3 (Shared pairs). *Let A and B be ground-truth communities, and let the pair $\{u_1, u_2\} \subseteq A \cap B$ lie in an admissible triple $T_A \subseteq A$ and in an admissible triple $T_B \subseteq B$. Then T_A and T_B lie in the same component of L_t, whatever the commonality between $A \setminus B$ and $B \setminus A$.*

Two communities that share a pair of nearby nodes are thus read as one, much as k-clique percolation reads communities sharing $k-1$ nodes as one [33]. The random merges of our benchmarks are not isolated: with $o_n = 0.05$, between 6% and 61% of the rings of a family are fused with another just above the lower edge of the window (Table S5).

Corollary 1 (Pair-disjointness). *At every threshold t, all admissible triples that contain a given pair of nodes lie in one component of L_t. In particular, an edge that lies in an admissible triple is assigned to exactly one community, and the communities of a node are those of the admitted edges incident to it.*

No pair of nodes binds two communities, so a node's memberships can be read off its admitted edges, as in link communities [40, 42]. This shares an assumption of clique percolation and link clustering that ground truth contradicts in many social and collaboration networks, where overlaps are more densely connected than the rest of the communities [59, 60]; in co-purchasing networks, by contrast, overlaps are small [61].

C. Cost

Admissibility is decided for each of the $W = \sum_v \binom{d_v}{2}$ wedges from three pair commonalities, all counted in one pass over the wedges, so the time and memory are

$$O(n + m + W) = O\big(n[1 + \langle d^2 \rangle]\big) \tag{6}$$

for n nodes and m edges. For a power-law degree tail of exponent γ this is $O(n)$ when $\gamma > 3$ and $O(n+m\log m)$ at $\gamma = 3$, the exponent of the Barabási–Albert model [62]; for $2 < \gamma < 3$ it grows as a power of n set by the largest degree, and how often real tails are that heavy is debated [63, 64].

In practice the heavy tail matters less than W suggests. Since $J(u,v) \le \rho(u,v) \equiv (\min(d_u, d_v) + 1)/(\max(d_u, d_v) + 1)$, a wedge containing an edge with $\rho < t_{\min}$ cannot be admitted at any $t \ge t_{\min}$ and can be discarded before scoring, provided the remaining commonalities are computed from the complete neighbourhoods (Sec. S3). The excess wedges of a heavy tail are exactly of this kind: they join a hub to neighbours of far smaller degree. On thirty-two SNAP networks with up to 4×10^6 nodes, $W/(m \ln m)$ ranges from 0.15 on road networks to 268 on a web graph with hubs of degree 8.4×10^4, but after pruning at $t_{\min} = 0.02$ it is at most 12 on all of them (Table S19); on YouTube it falls from 33 to 3.7. The cost is therefore that of the $\gamma = 3$ case even where W itself is not. Once the triple scores are stored, every further threshold costs $O(W)$, so a whole sweep of the threshold costs one scoring, whereas a sweep of Leiden's resolution or Infomap's Markov time costs a full run per value. Clique percolation is the costly method: the number of maximal cliques can grow exponentially with n [65], and on one of the fourteen smaller networks of Sec. VII E it is out of reach.

VI. BENCHMARKS AND EVALUATION PROTOCOL

A. Benchmarks

Planted overlapping ring lattices. Following Refs. [17, 66], each planted community is a ring lattice of s nodes and degree k, and communities may differ in s and k. Overlap is planted by merging $\lfloor o_n n_0 \rfloor$ pairs of nodes from different rings (n_0 nodes before the merges); the survivor belongs to both rings and keeps both neighbourhoods. Noise rewires a fraction $p \in \{0, 0.05, \ldots, 0.8\}$ of the original edges, each keeping one endpoint and moving the other to a non-neighbour chosen uniformly or preferentially. The families (Table S3) range from uniform rings ($n_0 = 2000$, $(s,k) \in \{(50,8),(100,16)\}$) through "two worlds" (five dense rings of 200 nodes, twenty sparse rings of 50) and "three big, many small" to a gradient of ten rings from 600 to 30 nodes; at $o_n \in \{0, 0.05, 0.10\}$, with four settings under preferential noise, they give 28 ring settings. Numbers quoted for a family without its overlap level are at $o_n = 0.05$. Further families each isolate one question: *isolated-merge* families place the merges within the hypothesis of Theorem 2; *geometry* families rewire a fraction q of each ring's edges inside the ring, moving each community from a lattice ($q = 0$) to a random graph ($q = 1$) at fixed sizes, degrees and overlap; and controls add hubs, split the edges of shared nodes between their rings as in LFR graphs, use random geometric communities, or have no planted communities at all (a random geometric graph and a Holme–Kim graph [67]). Every setting is in Sec. S2 of the Supplemental Material.

Overlapping LFR graphs. We use the reference generator [68, 69] with $N = 2000$, a fraction $o_n \in \{0, 0.1, 0.3\}$ of nodes in $o_m = 2$ communities and mixing $\mu \in \{0.1, \ldots, 0.6\}$, plus five families at $o_n = 0.1$ that change the degree, o_m, the community sizes or the clustering.

B. Methods compared

Three *ablations* each remove one ingredient: *triangles only* admits only closed triples (J-thresholded 3-clique percolation); *two-test* admits an open wedge when its two edges pass, without testing its non-adjacent pair; and *edge-connected* joins triples only through shared

edges. The fourteen other methods are link communities [40] with the same similarity, cut at every value of HyperCommon's threshold grid; clique percolation (CPM) [33]; SCAN [27] with cosine and with Jaccard similarity (SCAN-J); Leiden [6] with the Reichardt–Bornholdt modularity [70] and with the Constant Potts Model [49] (L-CPM); Infomap [7] over a range of Markov times [71, 72]; Walktrap [73]; label propagation (LPA) [11]; SLPA [13]; DEMON [74]; ANGEL [75]; OSLOM [10], which keeps a group only if it is significant against a configuration-model null, with its significance level as the parameter; and the nested degree-corrected stochastic block model (SBM) [8], fitted with GRAPH-TOOL [76], each level of its hierarchy treated as a parameter value. OSLOM and the SBM were run on the ring and LFR benchmarks, the random-geometric controls and the null graphs, and OSLOM also on all real networks. BigClam [77], ego-splitting [78] and the local-fitness method LFM [79] are not included (LFM took more than ten minutes on a single 2000-node graph, against the thousands of graphs of the benchmark). Every grid (Table S4) was widened and fixed after a preliminary run and before any reported result was computed.

C. Protocols

Every method, HyperCommon included, is scored under the same rules. *Oracle*: on each test graph each method is credited with its best parameter, which measures what its family of covers can achieve. *Label-free*: each method takes the parameter whose cover has the largest overlapping modularity [45],

$$Q_{\mathrm{ov}} = \frac{1}{2m}\sum_{c}\sum_{u,v\in c}\frac{1}{O_u O_v}\Big(A_{uv} - \frac{d_u d_v}{2m}\Big), \qquad (7)$$

where O_u is the number of communities that contain u and nodes in no community contribute nothing; for a partition this is the Newman–Girvan modularity. Two further protocols check these two: *native*, each method at its own default or selection rule where it has one (Leiden at $\gamma = 1$, L-CPM at the edge density, link communities at the largest partition density, Infomap at Markov time 1, SLPA at $r = 0.1$, OSLOM at its default level 0.1, the SBM at its finest level), and *frozen*, one parameter per family and method fixed on separate calibration graphs.

D. Metrics

Covers are compared with the Omega index [80]. For a cover $\mathcal{C}$ let $c_{\mathcal{C}}(i,j)$ be the number of its communities containing both i and j, and $\pi_{\mathcal{C}}(c)$ the fraction of pairs with $c_{\mathcal{C}} = c$; with ground truth $\mathcal{G}$, $\Omega = (\Omega_u - \Omega_e)/(1-\Omega_e)$, where $\Omega_u = \Pr[c_{\mathcal{C}} = c_{\mathcal{G}}]$ and $\Omega_e = \sum_c \pi_{\mathcal{C}}(c)\pi_{\mathcal{G}}(c)$. Omega counts node pairs and is dominated by the largest communities, so we also report the overlapping normalised mutual information (ONMI) [81], a per-community recovery score (the mean Jaccard similarity of each true community with its best match), the best-match F1 score [52] on real networks, and coverage. Unassigned nodes are penalised by Omega and ONMI. Each metric is averaged over the noise range per test seed; uncertainties are 95% bootstrap intervals over ten test seeds, and differences between methods are paired by seed.

VII. RESULTS

A. Exact recovery and the threshold

An exact implementation confirms Theorem 2. On rings whose merges satisfy its isolation hypothesis ($n_0 = 2000$, $o_n = 0.01$), without noise HyperCommon returns the planted rings exactly at each of the 20 thresholds of our grid inside the window for $k = 8$ and the 22 inside it for $k = 16$, and at no threshold outside it ($\Omega < 0.98$). With a fifth of the edges rewired, the mean Ω at the best threshold is still 1.000 for $k = 16$ and 0.991 for $k = 8$. Without the third-pair test the same graphs are never recovered exactly: the two edges of the wedge that crosses a merge have commonality $J_u(1)$, at least every threshold of the window, so the pair rule admits that wedge throughout it.

When merges fall at random they are rarely isolated: at $o_n = 0.05$ a ring carries on average at least as many merge positions as fit on it more than k apart, and rings fuse through nearby merged nodes (Proposition 3). The noise-free curve still stays between 0.92 and 0.99 across the window for $k = 8$ [Fig. 3(a)]. As noise grows the best threshold moves down towards $t \approx 0.1$, so the threshold has to be chosen per graph. The overlapping modularity of the cover chooses it well: the label-free threshold attains at least 95% of the best threshold's mean Ω on 27 of the 36 benchmark settings and at least 90% on 31 (Tables S6 and S8).

B. Planted rings

Table I summarises the benchmarks. Without labels, over the 28 ring settings HyperCommon has the highest Ω in 15, by paired 95% intervals that exclude zero, is level with the best other method in 6 and trails in 7: on dense rings with little overlap, where Leiden leads (0.84 against 0.78 at $k = 16$, $o_n = 0$), on rings of equal size and unequal degree at $o_n = 0.10$, and on the gradient, where link communities lead. The margins are largest where community sizes differ and communities overlap: $\Omega = 0.78$ against 0.68 for clique percolation on three big, many small, and 0.76 against 0.66 on two worlds [Fig. 4(c,d)].

Part of these margins lies in the selector rather than in the objectives. At its best resolution the Constant Potts

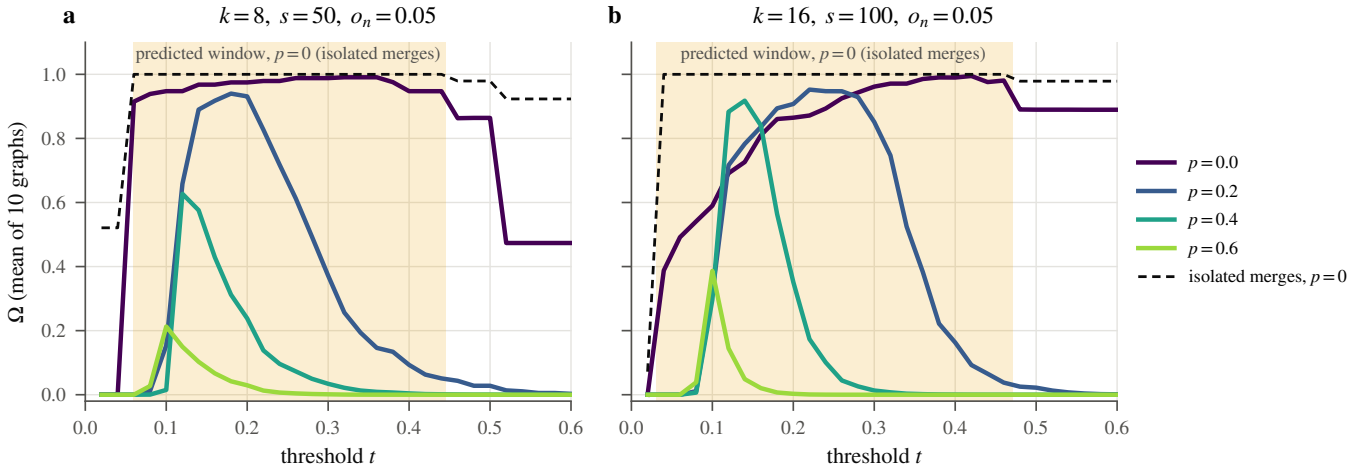


FIG. 3. Accuracy of HyperCommon as a function of the threshold on overlapping uniform rings ($n_0 = 2000$, $o_n = 0.05$), one curve per rewiring level p, each the mean over ten test graphs. Shaded: the exact-recovery window that Theorem 2 predicts for $p = 0$ when merges are isolated; the plotted graphs' random merges do not satisfy the isolation hypothesis. Black dashed: graphs with isolated merges ($o_n = 0.01$, merges more than $2k$ apart) at $p = 0$, which reach $\Omega = 1$ at every grid threshold inside the window and at none outside it.

Model reaches 0.71 on three big, many small and 0.78 on two worlds, against 0.34 and 0.64 when modularity chooses its resolution, while HyperCommon goes from 0.80 to 0.78 and from 0.78 to 0.76. Two checks bound this effect. At each method's own default or selection rule (native protocol), HyperCommon is still ahead in 17 of the 28 settings, although the Constant Potts Model at the edge density is then level with it on two worlds with overlap. With the best parameter on every graph (oracle), every method narrows the gap: HyperCommon is ahead in 11, level in 4 and behind in 13, mostly behind one of the two Leiden objectives, and stays ahead on three big, many small at every overlap level (0.80 against 0.72 for link communities) and on the gradient with overlap. Modularity with a single resolution fails on the gradient even without noise ($\Omega = 0.28$ for Leiden, against 0.99 for HyperCommon), because small rings are merged while large ones are split [47, 82]. With one parameter frozen per family, HyperCommon leads in only 1 setting: its best threshold moves with noise, so one threshold fits the noise-free graphs or the noisy ones but not both. On the gradient the label-free selector errs the other way, picking too fine a cover (181 communities against 147 at the best threshold and 10 planted).

Omega is dominated by the largest communities. On two worlds and three big, many small, HyperCommon recovers the large rings best but splits the small sparse ones, which the Constant Potts Model recovers best (0.53–0.54 against 0.33–0.38; Tables S11 and S12), so with labels the Constant Potts Model leads there by ONMI. Over the 28 ring settings HyperCommon leads by ONMI on 10 with labels and on 15 without (Tables S9 and S10). Without labels its accuracy also decays latest with noise, or within 0.01 of the latest, in ten of the sixteen ring families (Table S7).

C. Ablations

Testing the third, non-adjacent pair of a wedge matters most: without it Ω falls in all 36 benchmark settings under the oracle and the label-free protocol, by 0.06–0.34 on rings and up to 0.42 on LFR graphs, because a wedge whose two edges pass reaches into another community whenever its centre has a neighbour there. Admitting open wedges improves on triangles alone in 26 and 25 of the 36 settings under the two protocols, most where triangles are scarce (0.04–0.11 on sparse rings). Joining triples only through shared edges changes Ω by at most 0.02 on the rings when the parameter is chosen per graph and is slightly better on most LFR families; we keep joins through non-adjacent pairs because they give Corollary 1. Link communities, whose edge similarity is the commonality of the wedge's third pair, are less accurate than HyperCommon in 25 of the 28 ring settings under both protocols; the edge tests separate the two most where communities overlap. Among the commonality functions, Jaccard is the best, or within 0.01 of it, on seventeen of eighteen benchmark settings with the best threshold (Tables S17 and S18).

D. Where it breaks: LFR graphs and random communities

HyperCommon trails on all eight LFR families under every protocol; OSLOM is the most accurate method on most of them without labels. The pairwise signal is not what fails: the AUC of the commonality of edges inside against across planted communities is 0.97 at $\mu = 0.3$, and wedges spanning two communities pass the three-pair test at $t = 0.1$ with probability below 10^{-4}. What

TABLE I. Scorecard of the benchmarks under the four protocols: Ω of HyperCommon minus that of the best other method (ablations excluded), mean over the noise range ($p \in [0, 0.8]$ for rings, $\mu \in [0.1, 0.6]$ for LFR), ten test graphs per noise level, with the best other method in brackets when HyperCommon is not significantly ahead of it and its difference rounds to 0.00 or below. Mark: the paired 95% bootstrap interval of the difference over the ten test seeds lies above zero (+), contains zero (=) or lies below zero (−); intervals and the second-best method in Table S14. Oracle: best parameter per graph; label-free: largest overlapping modularity, Eq. (7); frozen: one parameter per family from calibration graphs; native: each method's own default or selection rule where it has one (Sec. VI), the label-free choice otherwise. Counts are not corrected for multiplicity. Method abbreviations as in Table S6; the full tables are Tables S6 and S8 (frozen and native in the repository).

Family	o_n	oracle	label-free	frozen	native
uniform, k=8	0.00	-0.05^{-} (Leiden)	$-0.00^{=}$ (Leiden)	-0.08^{-} (Leiden)	$-0.00^{=}$ (Leiden)
	0.05	$+0.00^{+}$	$+0.02^{+}$	-0.03^{-} (Leiden)	$+0.06^{+}$
	0.10	$+0.01^{+}$	$+0.05^{+}$	$+0.01^{=}$	$+0.06^{+}$
uniform, k=16	0.00	-0.08^{-} (Leiden)	-0.06^{-} (Leiden)	-0.10^{-} (Leiden)	-0.06^{-} (Leiden)
	0.05	-0.04^{-} (Leiden)	-0.03^{-} (Leiden)	-0.19^{-} (Leiden)	-0.04^{-} (Leiden)
	0.10	$-0.01^{=}$ (Leiden)	$+0.00^{=}$ (Leiden)	-0.32^{-} (Leiden)	$+0.00^{=}$ (Leiden)
equal k, unequal s, k=8	0.00	-0.03^{-} (Leiden)	$+0.01^{+}$	-0.04^{-} (Leiden)	$+0.01^{+}$
	0.05	$+0.01^{+}$	$+0.03^{+}$	$+0.00^{=}$ (Link)	$+0.09^{+}$
	0.10	$+0.00^{=}$ (Link)	$+0.04^{+}$	$+0.00^{=}$ (CPM)	$+0.08^{+}$
equal k, unequal s, k=16	0.00	-0.07^{-} (Leiden)	-0.04^{-} (Leiden)	-0.10^{-} (Leiden)	-0.04^{-} (Leiden)
	0.05	-0.04^{-} (Leiden)	$+0.00^{=}$ (Leiden)	-0.17^{-} (Leiden)	$+0.01^{+}$
	0.10	$+0.01^{+}$	$+0.02^{=}$	-0.31^{-} (Leiden)	$+0.04^{+}$
equal s, unequal k	0.00	-0.03^{-} (Leiden)	$-0.01^{=}$ (L-CPM)	-0.07^{-} (Leiden)	$+0.00^{=}$ (Leiden)
	0.05	-0.01^{-} (L-CPM)	$-0.01^{=}$ (L-CPM)	-0.18^{-} (Leiden)	$+0.02^{+}$
	0.10	-0.03^{-} (L-CPM)	-0.02^{-} (L-CPM)	-0.33^{-} (L-CPM)	$+0.03^{+}$
two worlds	0.00	-0.00^{-} (L-CPM)	$+0.03^{+}$	-0.05^{-} (L-CPM)	$+0.01^{+}$
	0.05	-0.01^{-} (L-CPM)	$+0.10^{+}$	-0.25^{-} (L-CPM)	$+0.00^{=}$ (L-CPM)
	0.10	$-0.00^{=}$ (L-CPM)	$+0.06^{+}$	-0.38^{-} (L-CPM)	$+0.01^{=}$
three big, many small	0.00	$+0.01^{+}$	$+0.03^{+}$	$+0.03^{+}$	$+0.09^{+}$
	0.05	$+0.07^{+}$	$+0.10^{+}$	-0.16^{-} (CPM)	$+0.10^{+}$
	0.10	$+0.04^{+}$	$+0.06^{+}$	-0.23^{-} (CPM)	$+0.06^{+}$
gradient	0.00	-0.01^{-} (Link)	-0.19^{-} (Link)	$-0.00^{=}$ (Link)	-0.14^{-} (CPM)
	0.05	$+0.03^{+}$	-0.16^{-} (Link)	$-0.02^{=}$ (ANGEL)	-0.11^{-} (ANGEL)
	0.10	$+0.03^{+}$	-0.15^{-} (Link)	-0.14^{-} (DEMON)	-0.12^{-} (CPM)
uniform k=8, α=1	0.05	$+0.01^{+}$	$+0.04^{+}$	-0.03^{-} (Leiden)	$+0.08^{+}$
uniform k=8, α=2	0.05	$+0.01^{+}$	$+0.02^{+}$	-0.02^{-} (Leiden)	$+0.08^{+}$
two worlds, α=1	0.05	-0.05^{-} (L-CPM)	$+0.11^{+}$	-0.29^{-} (L-CPM)	$+0.12^{+}$
two worlds, α=2	0.05	$+0.00^{=}$ (L-CPM)	$+0.13^{+}$	-0.20^{-} (L-CPM)	$+0.15^{+}$
LFR, o_n=0		-0.14^{-} (L-CPM)	-0.16^{-} (Info.)	-0.18^{-} (Leiden)	-0.16^{-} (Info.)
LFR, o_n=0.1		-0.23^{-} (OSLOM)	-0.25^{-} (OSLOM)	-0.33^{-} (OSLOM)	-0.27^{-} (OSLOM)
LFR, o_n=0.3		-0.40^{-} (OSLOM)	-0.39^{-} (OSLOM)	-0.58^{-} (OSLOM)	-0.42^{-} (OSLOM)
LFR, $\langle k \rangle$=10		-0.33^{-} (Leiden)	-0.34^{-} (OSLOM)	-0.37^{-} (Leiden)	-0.35^{-} (OSLOM)
LFR, o_m=4		-0.19^{-} (OSLOM)	-0.14^{-} (OSLOM)	-0.25^{-} (OSLOM)	-0.19^{-} (OSLOM)
LFR, sizes 50–250		-0.49^{-} (OSLOM)	-0.47^{-} (OSLOM)	-0.57^{-} (OSLOM)	-0.47^{-} (OSLOM)
LFR, C_{LFR}=0.3		-0.23^{-} (Leiden)	-0.23^{-} (SLPA)	-0.31^{-} (Leiden)	-0.26^{-} (Info.)
LFR, C_{LFR}=0.5		-0.08^{-} (Leiden)	-0.11^{-} (LPA)	-0.14^{-} (SLPA)	-0.12^{-} (SLPA)
+/=/−, rings		11/4/13 of 28	15/6/7 of 28	1/5/22 of 28	17/5/6 of 28
+/=/−, LFR		0/0/8 of 8	0/0/8 of 8	0/0/8 of 8	0/0/8 of 8

fails is connectivity inside large communities. LFR communities are random graphs internally, with degree independent of size, so a large community is sparse and its members share few neighbours: at $\mu = 0.3$, 89% of the wedges inside communities of at most 40 nodes pass at $t = 0.1$, but only 21% of those inside communities of at least 80 nodes. Percolation then breaks large communities into pieces at any threshold that keeps communities apart, which is why the deficit is largest with large communities (0.42 against 0.89 for OSLOM).

The geometry families test this directly (Table II): sizes, degrees and overlap are fixed while a fraction q of each ring's edges is rewired inside the ring, so adjacent members go from sharing up to $k-2$ neighbours to sharing about $(k-1)^2/s$. On lattice communities HyperCommon is ahead of every other method with labels in three of the four families and level in the fourth. With labels the advantage is gone once a tenth of each community's edges are rewired, while the communities are still strongly clustered; without labels it holds on three big, many small at $q = 0.1$; from $q = 0.3$ on the best other method is ahead everywhere. Degree-preserving swaps

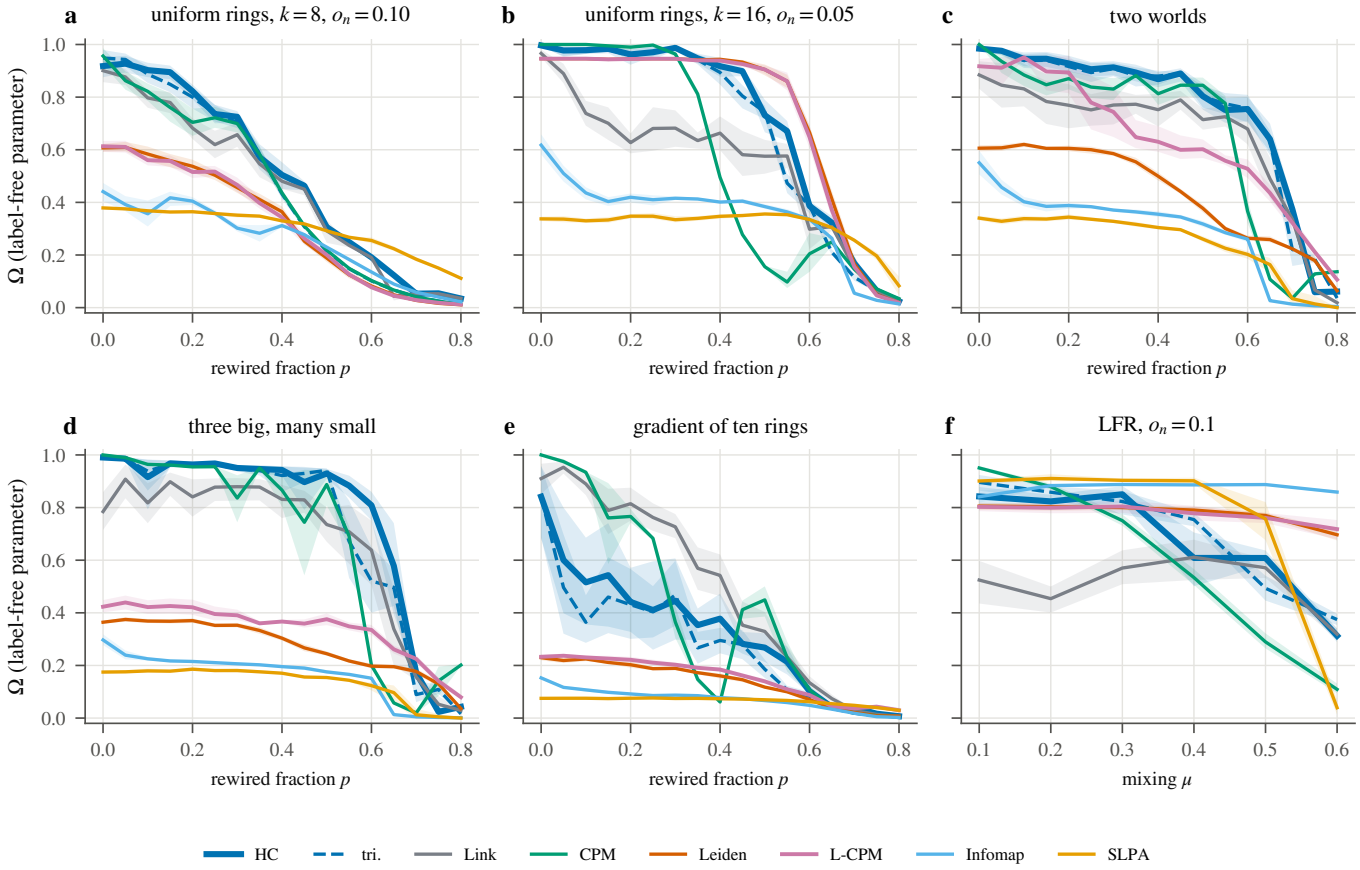


FIG. 4. Accuracy against noise with every method's parameter chosen without labels (largest overlapping modularity on each graph); lines are means over ten test graphs and bands 95% bootstrap intervals. (a,b) Uniform rings; (c–e) rings of unequal size and density ($o_n = 0.05$); (f) overlapping LFR graphs, against the mixing parameter. In (e) the selector picks too fine a cover for HyperCommon (Sec. VII B). OSLOM and the stochastic block model are not drawn; on the family of (f) OSLOM is the most accurate method (Table S6). The same panels under the oracle protocol are in Fig. S4.

give the same result within 0.01 at $q = 0.1$ and 0.3, so the change of degrees is not the reason. The controls (Table S15) point the same way: hubs inside the rings move HyperCommon's Ω by at most 0.03, splitting the edges of shared nodes as in LFR graphs by at most 0.05, and random geometric communities, clustered but not lattices, leave it behind the best other method with labels. On clustered graphs with no planted communities every method reports communities, HyperCommon 101 and 175 (Table S16), so a cover it returns is not by itself evidence of community structure.

E. Real networks

Table III reports detection on the full Amazon, DBLP and YouTube graphs [52]. HyperCommon's threshold is chosen without labels at little cost: the label-free F1 is 99.9%, 94% and 89% of its best F1. The same selector serves the modularity-based methods less well: Leiden falls to 0.53, 0.29 and 0.19 in F1, about half of its best values. Under this selector Infomap is the most accurate on all three graphs (F1 0.90, 0.52 and 0.34), with label-free HyperCommon within 0.03–0.05; by Ω the modularity- and flow-based partitions lead.

Table S20 of the Supplemental Material extends the comparison to fourteen smaller networks from the Netzschleuder repository [83]: ten Facebook ego networks with their users' own social circles [84], college football conferences [85], political books [86, 87] and blogs [88], and the classes of a primary school [89]. With its best threshold HyperCommon is the most accurate on college football and within 0.01 of the best, OSLOM, on political blogs; its mean rank among seventeen methods is 8.9 without labels and 8.1 with labels, and the most accurate method varies from network to network [14]. The choice of commonality function matters here: on the primary school it alone raises F1 from 0.41 to 0.83. Testing the third pair helps less than on the benchmarks: with the best threshold the two-test variant has the higher F1 on nine of the fourteen networks. Over all seventeen networks the label-free threshold retains a median 94% of the best F1.

TABLE II. Internal geometry: before the noise, a fraction q of each ring's edges is rewired inside the ring, from lattice rings ($q = 0$) to rings that are random graphs ($q = 1$), at fixed size and mean degree; $o_n = 0.05$. Uniform: 40 rings of 50 nodes, $k = 8$; two worlds: five rings of 200 nodes ($k = 32$) and twenty of 50 ($k = 8$); three big, many small: three of 400 ($k = 32$) and sixteen of 50 ($k = 8$); gradient: ten rings of 600 to 30 nodes ($k = 16$ to 8). Mean Ω over $p \in \{0, 0.1, \ldots, 0.6\}$, ten test graphs per noise level, under the oracle and label-free protocols, for HyperCommon (HC), its triangles-only variant and the best other method; C: mean clustering of the noiseless graphs (ten seeds); cov.: coverage of HC's label-free cover. Superscripts on HC: paired 95% interval of its difference from the best other method excludes zero ($^+$ HC ahead, $^-$ behind). Bold: HC best or within 0.005 of the best other method. Means are over $p \in \{0, 0.1, \ldots, 0.6\}$, so the $q = 0$ entries differ from those of Tables S6 and S8 for the same families, which average over $p \in [0, 0.8]$.

			oracle			label-free			
Family	q	C	HC	tri.	best other	HC	tri.	best other	cov.
uniform, k=8	0.0	0.62	**0.73**$^+$	0.65	0.73 (Link)	**0.70**$^+$	0.64	0.67 (Link)	0.98
	0.1	0.48	0.68$^-$	0.56	0.81 (Leiden)	0.65	0.56	0.66 (Leiden)	0.97
	0.3	0.28	0.62$^-$	0.41	0.85 (Leiden)	0.57$^-$	0.41	0.74 (Infomap)	0.95
	0.6	0.16	0.61$^-$	0.28	0.86 (Leiden)	0.55$^-$	0.28	0.77 (Infomap)	0.96
	1.0	0.15	0.60$^-$	0.29	0.85 (Leiden)	0.55$^-$	0.28	0.78 (Infomap)	0.95
two worlds	0.0	0.67	**0.91**	0.90	0.91 (L-CPM)	**0.89**$^+$	0.89	0.82 (CPM)	0.94
	0.1	0.51	0.88$^-$	0.87	0.94 (L-CPM)	0.86$^-$	0.86	0.91 (L-CPM)	0.93
	0.3	0.30	0.84$^-$	0.81	0.95 (L-CPM)	0.82$^-$	0.80	0.92 (L-CPM)	0.94
	0.6	0.17	0.72$^-$	0.68	0.95 (L-CPM)	0.69$^-$	0.66	0.92 (L-CPM)	0.91
	1.0	0.16	0.66$^-$	0.66	0.95 (L-CPM)	0.62$^-$	0.66	0.92 (L-CPM)	0.92
three big, many small	0.0	0.68	**0.94**$^+$	0.93	0.87 (CPM)	**0.93**$^+$	0.91	0.85 (CPM)	0.90
	0.1	0.51	0.92$^-$	0.91	0.95 (L-CPM)	**0.91**$^+$	0.91	0.83 (CPM)	0.91
	0.3	0.28	0.84$^-$	0.80	0.96 (L-CPM)	0.83$^-$	0.79	0.95 (L-CPM)	0.91
	0.6	0.13	0.51$^-$	0.61	0.96 (L-CPM)	0.50$^-$	0.56	0.95 (L-CPM)	0.83
	1.0	0.11	0.28$^-$	0.54	0.96 (L-CPM)	0.27$^-$	0.45	0.95 (L-CPM)	0.80
gradient of ten rings	0.0	0.66	**0.74**$^+$	0.72	0.70 (Link)	0.42$^-$	0.37	0.64 (Link)	0.93
	0.1	0.49	0.65$^-$	0.63	0.73 (L-CPM)	0.43$^-$	0.38	0.55 (Link)	0.93
	0.3	0.25	0.44$^-$	0.43	0.86 (L-CPM)	0.40$^-$	0.34	0.76 (Walktrap)	0.93
	0.6	0.08	0.16$^-$	0.11	0.88 (L-CPM)	0.16$^-$	0.09	0.86 (L-CPM)	0.87
	1.0	0.04	0.11$^-$	0.02	0.87 (L-CPM)	0.09$^-$	0.01	0.85 (L-CPM)	0.89

VIII. DISCUSSION

HyperCommon reads communities off local evidence only: a pair's shared neighbourhood, required of all three pairs of a wedge and propagated through shared pairs, never compared with a global null model. On ring lattices commonality does not depend on community size (Lemma 1), so one threshold suits rings of thirty and of six hundred nodes alike, where no single resolution of either Leiden objective keeps the large rings whole and the small ones apart. Theory and benchmarks meet at the exact-recovery window, and the ablations support each test of the triple rule: open wedges help where triangles are scarce, the non-adjacent pair keeps overlapping communities apart, and the edge tests separate HyperCommon from link communities.

The method's reach is set by geometry. It leads while communities are lattice-like, so that adjacent members share many neighbours, and trails once a tenth (with labels) or three tenths (without) of each ring's edges are rewired internally, on all LFR families and on random geometric communities with labels. The same property that makes overlap clean, that no pair binds two communities, means that communities joined by an admissible triple are read as one (Proposition 3), which limits the method where overlaps are densely connected [59, 60].

Three limits qualify these results. First, part of the label-free margin reflects the selector: modularity costs the Constant Potts Model more than HyperCommon on unequal rings, and with each method's best parameter HyperCommon leads on fewer settings (11 of 28) than it trails (13). Second, on real networks HyperCommon is competitive but rarely the most accurate, the gains from the choice of function rest on labels, and the two-test variant, without the third-pair test, has the higher F1 on most of the smaller networks; none of the real networks tested is known to have lattice-like groups. Third, like every method we tested, it reports many communities on clustered graphs with none planted, so a test against randomised graphs that keep the degrees and the clustering would have to be added.

Two directions follow. The label-free criterion is global and, on the gradient, selects too fine a cover; since the covers at all thresholds form one hierarchy read off a single scoring of the wedges (Sec. V C), thresholds could be chosen separately on its branches, for instance by the cluster-stability criterion of HDBSCAN [90]. And the commonality function is a second degree of freedom: ev-

TABLE III. Detection on the full SNAP Amazon (334 863 nodes), DBLP (317 080) and YouTube (1 134 890) graphs, scored on the nodes of their ground-truth communities (top 5000 for Amazon and DBLP, all groups of at least three nodes for YouTube), to which every detected community is restricted. Best-match F1 against the recorded communities and Ω against their distinct node sets, at each method's best parameter by that metric (best) and at the parameter chosen without labels by maximal overlapping modularity on the full graph (free); cov.: fraction of the scored nodes in at least one community at the free parameter. HC_f: the seven commonality functions together, the free choice taken over all of them. Leiden at $\gamma = 1$ and Infomap at Markov time 1 are the two methods at their defaults (one parameter, so best and free coincide). Connected components are a trivial reference. On YouTube HyperCommon uses degree-ratio pruning, exact for every $t \geq 0.02$ (Sec. V C), which excludes link communities and the other functions. Clique percolation and SLPA were run only on the subgraphs induced by the labelled nodes, and SCAN, Walktrap, DEMON and ANGEL not at all, to bound the computing time of the large runs. Stochastic methods: one seed. Scored against the distinct node sets of the recorded communities instead, F1 changes by at most 0.03, and OSLOM moves above two of the HyperCommon variants on Amazon (both in the repository). The top-5000 lists of Amazon and DBLP frequently consist of near-cliques [32], which can favour methods that find small dense groups. Bold: best in column.

	Amazon					DBLP					YouTube				
	F1 best	F1 free	Ω best	Ω free	cov.	F1 best	F1 free	Ω best	Ω free	cov.	F1 best	F1 free	Ω best	Ω free	cov.
components	0.020	0.020	0.000	0.000	1.00	0.075	0.075	0.000	0.000	1.00	0.060	0.060	0.000	0.000	1.00
HC	0.856	0.854	0.459	0.459	0.99	0.516	0.486	0.024	0.024	0.94	0.322	0.289	0.017	−0.007	0.67
HC_f	0.884	0.852	0.477	0.477	1.00	0.520	0.486	0.037	0.024	0.94	–	–	–	–	–
e-conn.	0.857	0.857	0.485	0.485	0.99	0.516	0.487	0.024	0.024	0.94	0.323	0.292	0.016	−0.007	0.67
tri.	0.840	0.838	0.415	0.415	0.95	0.513	0.485	0.030	0.019	0.89	0.309	0.300	0.015	0.006	0.49
2-test	0.887	0.887	0.424	0.424	0.95	0.507	0.459	0.014	0.014	0.83	0.280	0.215	0.005	−0.011	0.53
Link	0.842	0.827	0.487	0.487	1.00	0.490	0.446	0.031	0.024	1.00	–	–	–	–	–
Leiden	0.924	0.534	0.600	0.392	1.00	0.542	0.285	**0.154**	**0.152**	1.00	**0.363**	0.192	**0.142**	0.074	1.00
Leiden, γ=1	0.534	0.534	0.392	0.392	1.00	0.285	0.285	0.152	**0.152**	1.00	0.192	0.192	0.074	0.074	1.00
L-CPM	0.929	0.561	**0.617**	0.385	1.00	0.532	0.237	0.150	0.150	1.00	0.337	0.252	0.113	0.104	1.00
Infomap	**0.935**	0.900	0.583	**0.583**	1.00	0.545	0.518	0.042	0.029	1.00	0.355	**0.337**	0.126	**0.126**	1.00
Infomap, Markov time 1	0.918	**0.918**	0.488	0.488	1.00	0.545	**0.545**	0.013	0.013	1.00	0.337	**0.337**	0.126	**0.126**	1.00
LPA	0.820	0.820	0.373	0.373	1.00	0.508	0.508	0.010	0.010	1.00	0.216	0.216	0.007	0.007	1.00
OSLOM	0.882	0.834	0.560	0.536	1.00	**0.549**	0.351	0.028	0.028	1.00	0.338	0.253	0.125	0.125	1.00

ery member of the family carries the signal, but the most accurate one varies between networks, so learning the function and the threshold from a network's own structure is the natural next step.

DATA AND CODE AVAILABILITY

The implementation (including the vectorised code with the degree-ratio pruning used on YouTube), the benchmark generators, the script behind every number, table and figure, the raw results (one row per graph, method and parameter) and the per-setting paired bootstrap intervals are in release `v1.0.0` of https://github.com/hypercommon/hypercommon, under the MIT licence; `paper/README.md` lists the command that regenerates each table and figure, together with the software versions, the seeds and the machines on which the results were computed. The real networks are public: SNAP (https://snap.stanford.edu/data) and the Netzschleuder repository [83].

ACKNOWLEDGMENTS

D.D. acknowledges the support of the Friedrich and Ira Sachs scholarship at Bar-Ilan University.

Appendix A: Proofs

This appendix proves the results of Sec. V; the two ends of the hierarchy, the further ring-lattice windows, the fusion of rings and the cost estimate are in Sec. S3 of the Supplemental Material.

1. Determinism, locality and nesting

Proof of Proposition 1. $\mathcal{T}_t$ is defined pointwise from G and t, the adjacency of L_t pointwise from $\mathcal{T}_t$, and the connected components of a graph are unique. □

Proof of Proposition 2. Two triples are adjacent in L_t iff they share a pair $\{x, y\}$. If $\{x, y, w\}$ is a wedge its centre is x, y or w, and in each case w is adjacent to x or y. Every neighbour of a visited triple is therefore generated from one of its three pairs, so the search traverses the component and nothing else. Admissibility of $\{x, y, w\}$ depends on $N[x], N[y], N[w]$ only. □

Proof of Theorem 1. $J \geq t_2$ implies $J \geq t_1$ and the wedge condition does not involve t, so $\mathcal{T}_{t_2} \subseteq \mathcal{T}_{t_1}$. Adjacency (sharing two nodes) does not involve t either, so L_{t_2} is induced. A connected subgraph of L_{t_2} is connected in L_{t_1}, so each component of L_{t_2}, and the union of its triples, lies inside a single component of L_{t_1}. □

2. Ring lattices

Proof of Lemma 1. $N[x]$ is an arc of $k+1$ consecutive ring nodes, and two arcs at offset $d \le k$ overlap in $k+1-d$ nodes when $s \ge 2k+2$, which gives $J(d)$. A merged node's closed neighbourhood is the union of its two arcs, $2k+1$ nodes, and the arc of x meets only the one in its own ring, which gives $J_u(d)$. Nodes on different rings can share only merged nodes, and isolation leaves at most one, u, in both neighbourhoods. □

Proof of Theorem 2. No merges. J decreases with ring distance, and any three distinct ring nodes include a pair at distance at least two, so no triple is admissible when $t > J(2)$ and every ring is lost. For $t \le J(2)$ the consecutive triples $\{i, i+1, i+2\}$, with pair distances $1, 1, 2$, are admissible and each shares a pair with the next, so they form a single component covering the ring. Every other admissible wedge lies in a cyclic interval of length at most k, and moving its outer nodes inward preserves admissibility and connects it to a consecutive triple, so there are no further components. No wedge spans two rings, which share no edge, so the components are exactly the rings.

Lower edge. A wedge with nodes on two rings needs a node adjacent to both, i.e. a merged node u; its weakest pair is $J(z, y') = 1/(2k+1)$ by Eq. (3), the other two being of the form $J_u(d)$ with $d \le k/2$, which is larger. If $t \le 1/(2k+1)$, the triple $\{z, u, y'\}$ is admissible and, since $J_u(1)$ and $J(2)$ exceed $1/(2k+1)$ for $k \ge 4$, so are triples of both rings through the pairs $\{z, u\}$ and $\{u, y'\}$: the two rings fall into one component. If $t > 1/(2k+1)$ no admissible triple contains nodes of two rings other than u, so each component lies within one ground-truth community.

Upper edge. Above the lower edge every admissible triple lies within one ring, and each wedge lies in an arc of length at most k, which by isolation contains at most one merge position. Fix a merged node u at position 0 of one of its rings. Every admissible triple through u on this ring has two u-pairs, at distinct distances or on opposite sides of u, so the best is $T_u = \{-1, 0, 1\}$, with weakest pairs $J_u(1) = k/(2k+2)$: for $t > J_u(1)$, u is in no triple of the ring and is missing from its community. For $J_u(2) < t \le J_u(1)$, T_u is the only admissible triple through u on this ring; the corresponding triple on u's other ring shares only u with it and cannot attach it, so T_u connects to the ring only through its pair $\{-1, 1\}$. Every completion of that pair other than u has score at most $J(3) = (k-2)/(k+4)$, attained by ± 2. Hence for $\max(J(3), J_u(2)) < t \le J_u(1)$, T_u is an isolated component and $\{-1, 0, 1\}$ appears as a spurious community.

Conversely, let $t \le J_u(1)$ and $t \le \max(J(3), J_u(2))$. The consecutive triples that avoid u are admissible (their pairs are at distances 1, 1, 2, and $J(2) > J_u(1) \ge t$ for $k \ge 4$), and so is T_u. If $t \le J(3)$, the bridges $\{-2, -1, 1\}$ and $\{-1, 1, 2\}$, whose pairs avoid u and lie at distances 1, 2 and 3, are admissible; they share $\{-2, -1\}$ and $\{1, 2\}$ with the consecutive triples on either side of the merge and $\{-1, 1\}$ with T_u. If instead $t \le J_u(2)$, the consecutive triples $\{-2, -1, 0\}$ and $\{0, 1, 2\}$ through u are admissible and share $\{-1, 0\}$ and $\{0, 1\}$ with T_u. Either way the triples listed so far form one component per ring. Every other admissible triple T of the ring joins it through *contraction moves*: replace an outer node of T whose next node towards the other two (skipping u if $u \notin T$) is not already in T by that node, and stop when the three nodes are consecutive in the ring order (with u removed if $u \notin T$). A move keeps two nodes of T, so successive triples share a pair; it increases no ring distance within the triple, so the triple remains a wedge; and it does not decrease the score, because pairs not involving u have commonality $J(d)$ and pairs involving u have $J_u(d)$, both decreasing in d, and a move that replaces u itself turns each u-pair at distance d into a pair at distance $d' \le d$ with $J(d') \ge J(d) > J_u(d)$. The moves therefore end, through admissible triples, at a consecutive triple, a bridge, or a triple through u already listed, so all admissible triples within each ring are connected. Since $J_u(1) < J(2)$ for $k \ge 4$, the no-merge condition is implied, and Eq. (4) follows. □

Isolation. The proof uses isolation only through the facts that an arc of $k+1$ consecutive nodes contains at most one merge position and that nodes of different rings share at most one merged neighbour. For a ring pair merged twice the separation $k+1$ is sharp: shared nodes k apart on both rings raise the lower edge to $1/k$.

3. Shared pairs and pair-disjointness

Proof of Proposition 3. If $T_A \ne T_B$, the two triples share exactly the two nodes u_1, u_2 and are adjacent in L_t. □

Proof of Corollary 1. Two distinct admissible triples that contain the same pair share exactly two nodes and are therefore adjacent in L_t, so all admissible triples through the pair lie in one component. □

[1] S. Fortunato, Community detection in graphs, Physics Reports **486**, 75 (2010).

[2] S. Fortunato and M. E. J. Newman, 20 years of network community detection, Nature Physics **18**, 848 (2022).

[3] J. Xie, S. Kelley, and B. K. Szymanski, Overlapping community detection in networks: The state-of-the-art and comparative study, ACM Computing Surveys **45**, 43 (2013).

[4] S. Fortunato and D. Hric, Community detection in networks: A user guide, Physics Reports **659**, 1 (2016).

[5] M. E. J. Newman and M. Girvan, Finding and evaluating community structure in networks, Physical Review E **69**, 026113 (2004).

[6] V. A. Traag, L. Waltman, and N. J. van Eck, From Louvain to Leiden: guaranteeing well-connected communities, Scientific Reports **9**, 5233 (2019).

[7] M. Rosvall and C. T. Bergstrom, Maps of random walks on complex networks reveal community structure, Proceedings of the National Academy of Sciences **105**, 1118 (2008).

[8] T. P. Peixoto, Hierarchical block structures and high-resolution model selection in large networks, Physical Review X **4**, 011047 (2014).

[9] B. Ball, B. Karrer, and M. E. J. Newman, Efficient and principled method for detecting communities in networks, Physical Review E **84**, 036103 (2011).

[10] A. Lancichinetti, F. Radicchi, J. J. Ramasco, and S. Fortunato, Finding statistically significant communities in networks, PLoS ONE **6**, e18961 (2011).

[11] U. N. Raghavan, R. Albert, and S. Kumara, Near linear time algorithm to detect community structures in large-scale networks, Physical Review E **76**, 036106 (2007).

[12] S. Gregory, Finding overlapping communities in networks by label propagation, New Journal of Physics **12**, 103018 (2010).

[13] J. Xie, B. K. Szymanski, and X. Liu, SLPA: Uncovering overlapping communities in social networks via a speaker-listener interaction dynamic process, in *2011 IEEE 11th International Conference on Data Mining Workshops* (2011) pp. 344–349.

[14] L. Peel, D. B. Larremore, and A. Clauset, The ground truth about metadata and community detection in networks, Science Advances **3**, e1602548 (2017).

[15] A. Ghasemian, H. Hosseinmardi, and A. Clauset, Evaluating overfit and underfit in models of network community structure, IEEE Transactions on Knowledge and Data Engineering **32**, 1722 (2020).

[16] M. S. Granovetter, The strength of weak ties, American Journal of Sociology **78**, 1360 (1973).

[17] D. J. Watts and S. H. Strogatz, Collective dynamics of 'small-world' networks, Nature **393**, 440 (1998).

[18] T. P. Peixoto, Disentangling homophily, community structure, and triadic closure in networks, Physical Review X **12**, 011004 (2022).

[19] P. Jaccard, The distribution of the flora in the alpine zone, New Phytologist **11**, 37 (1912).

[20] E. A. Leicht, P. Holme, and M. E. J. Newman, Vertex similarity in networks, Physical Review E **73**, 026120 (2006).

[21] T. Zhou, L. Lü, and Y.-C. Zhang, Predicting missing links via local information, The European Physical Journal B **71**, 623 (2009).

[22] B. Barzel and A.-L. Barabási, Network link prediction by global silencing of indirect correlations, Nature Biotechnology **31**, 720 (2013).

[23] S. Feizi, D. Marbach, M. Médard, and M. Kellis, Network deconvolution as a general method to distinguish direct dependencies in networks, Nature Biotechnology **31**, 726 (2013).

[24] A. R. Benson, D. F. Gleich, and J. Leskovec, Higher-order organization of complex networks, Science **353**, 163 (2016).

[25] F. Battiston, G. Cencetti, I. Iacopini, V. Latora, M. Lucas, A. Patania, J.-G. Young, and G. Petri, Networks beyond pairwise interactions: Structure and dynamics, Physics Reports **874**, 1 (2020).

[26] F. Radicchi, C. Castellano, F. Cecconi, V. Loreto, and D. Parisi, Defining and identifying communities in networks, Proceedings of the National Academy of Sciences **101**, 2658 (2004).

[27] X. Xu, N. Yuruk, Z. Feng, and T. A. J. Schweiger, SCAN: A structural clustering algorithm for networks, in *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2007) pp. 824–833.

[28] R. A. Jarvis and E. A. Patrick, Clustering using a similarity measure based on shared near neighbors, IEEE Transactions on Computers **C-22**, 1025 (1973).

[29] J.-P. Onnela, J. Saramäki, J. Hyvönen, G. Szabó, D. Lazer, K. Kaski, J. Kertész, and A.-L. Barabási, Structure and tie strengths in mobile communication networks, Proceedings of the National Academy of Sciences **104**, 7332 (2007).

[30] A. Ghasemian, H. Hosseinmardi, A. Galstyan, E. M. Airoldi, and A. Clauset, Stacking models for nearly optimal link prediction in complex networks, Proceedings of the National Academy of Sciences **117**, 23393 (2020).

[31] C. E. Tsourakakis, J. Pachocki, and M. Mitzenmacher, Scalable motif-aware graph clustering, in *Proceedings of the 26th International Conference on World Wide Web* (2017) pp. 1451–1460.

[32] T. Chen and C. E. Tsourakakis, Parallel motif-based community detection, in *2024 IEEE International Conference on Big Data* (2024) pp. 509–518.

[33] G. Palla, I. Derényi, I. Farkas, and T. Vicsek, Uncovering the overlapping community structure of complex networks in nature and society, Nature **435**, 814 (2005).

[34] J. M. Kumpula, M. Kivelä, K. Kaski, and J. Saramäki, Sequential algorithm for fast clique percolation, Physical Review E **78**, 026109 (2008).

[35] I. J. Farkas, D. Ábel, G. Palla, and T. Vicsek, Weighted network modules, New Journal of Physics **9**, 180 (2007).

[36] J. Cohen, *Trusses: Cohesive subgraphs for social network analysis*, Tech. Rep. (National Security Agency, 2008).

[37] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu, Querying k-truss community in large and dynamic graphs, in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data* (2014) pp. 1311–1322.

[38] A. E. Sarıyüce, C. Seshadhri, A. Pınar, and Ü. V. Çatalyürek, Finding the hierarchy of dense subgraphs using nucleus decompositions, in *Proceedings of the 24th In-*

ternational Conference on World Wide Web (2015) pp. 927–937.
[39] Z. Zhang, L. Cui, Z. Pan, A. Fang, and H. Zhang, A triad percolation method for detecting communities in social networks, Data Science Journal **17**, 30 (2018).
[40] Y.-Y. Ahn, J. P. Bagrow, and S. Lehmann, Link communities reveal multiscale complexity in networks, Nature **466**, 761 (2010).
[41] Q. F. Lotito, F. Musciotto, A. Montresor, and F. Battiston, Hyperlink communities in higher-order networks, Journal of Complex Networks **12**, cnae013 (2024).
[42] T. S. Evans and R. Lambiotte, Line graphs, link partitions, and overlapping communities, Physical Review E **80**, 016105 (2009).
[43] T. S. Evans, Clique graphs and overlapping communities, Journal of Statistical Mechanics: Theory and Experiment **2010**, P12037 (2010).
[44] S. G. Aksoy, C. Joslyn, C. Ortiz Marrero, B. Praggastis, and E. Purvine, Hypernetwork science via high-order hypergraph walks, EPJ Data Science **9**, 16 (2020).
[45] H. Shen, X. Cheng, K. Cai, and M.-B. Hu, Detect overlapping and hierarchical community structure in networks, Physica A: Statistical Mechanics and its Applications **388**, 1706 (2009).
[46] B. Tóth, T. Vicsek, and G. Palla, Overlapping modularity at the critical point of k-clique percolation, Journal of Statistical Physics **151**, 689 (2013).
[47] S. Fortunato and M. Barthélemy, Resolution limit in community detection, Proceedings of the National Academy of Sciences **104**, 36 (2007).
[48] P. Ronhovde and Z. Nussinov, Local resolution-limit-free Potts model for community detection, Physical Review E **81**, 046114 (2010).
[49] V. A. Traag, P. Van Dooren, and Y. Nesterov, Narrow scope for resolution-limit-free community detection, Physical Review E **84**, 016114 (2011).
[50] D. Deinega, *Overlapping Communities Detection in Complex Networks*, Master's thesis, Ariel University (2023).
[51] W. W. Zachary, An information flow model for conflict and fission in small groups, Journal of Anthropological Research **33**, 452 (1977).
[52] J. Yang and J. Leskovec, Defining and evaluating network communities based on ground-truth, Knowledge and Information Systems **42**, 181 (2015).
[53] See Supplemental Material at [URL will be inserted by publisher] for the full benchmark tables under the four protocols; ONMI, per-community recovery, coverage and numbers of communities; the paired intervals behind the scorecard; the selector and grid-edge diagnostics; the control families and null graphs; the commonality family on the benchmarks; every method on the real networks; and the sources of numbers quoted in the text.
[54] C. Elkan and K. Noto, Learning classifiers from only positive and unlabeled data, in *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2008) pp. 213–220.
[55] D. Hric, R. K. Darst, and S. Fortunato, Community detection in networks: Structural communities versus ground truth, Physical Review E **90**, 062805 (2014).
[56] L. Lü and T. Zhou, Link prediction in complex networks: A survey, Physica A: Statistical Mechanics and its Applications **390**, 1150 (2011).
[57] G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval* (McGraw-Hill, 1983).
[58] E. Ravasz, A. L. Somera, D. A. Mongru, Z. N. Oltvai, and A.-L. Barabási, Hierarchical organization of modularity in metabolic networks, Science **297**, 1551 (2002).
[59] J. Yang and J. Leskovec, Structure and overlaps of ground-truth communities in networks, ACM Transactions on Intelligent Systems and Technology **5**, 26 (2014).
[60] J. Yang and J. Leskovec, Community-affiliation graph model for overlapping network community detection, in *2012 IEEE 12th International Conference on Data Mining* (2012) pp. 1170–1175.
[61] J. Yang and J. Leskovec, Overlapping communities explain core–periphery organization of networks, Proceedings of the IEEE **102**, 1892 (2014).
[62] R. Albert and A.-L. Barabási, Statistical mechanics of complex networks, Reviews of Modern Physics **74**, 47 (2002).
[63] A. D. Broido and A. Clauset, Scale-free networks are rare, Nature Communications **10**, 1017 (2019).
[64] I. Voitalov, P. van der Hoorn, R. van der Hofstad, and D. Krioukov, Scale-free networks well done, Physical Review Research **1**, 033034 (2019).
[65] J. W. Moon and L. Moser, On cliques in graphs, Israel Journal of Mathematics **3**, 23 (1965).
[66] I. Kanovsky, Small world models for social network algorithms testing, Procedia Computer Science **1**, 2341 (2010).
[67] P. Holme and B. J. Kim, Growing scale-free networks with tunable clustering, Physical Review E **65**, 026107 (2002).
[68] A. Lancichinetti, S. Fortunato, and F. Radicchi, Benchmark graphs for testing community detection algorithms, Physical Review E **78**, 046110 (2008).
[69] A. Lancichinetti and S. Fortunato, Benchmarks for testing community detection algorithms on directed and weighted graphs with overlapping communities, Physical Review E **80**, 016118 (2009).
[70] J. Reichardt and S. Bornholdt, Statistical mechanics of community detection, Physical Review E **74**, 016110 (2006).
[71] M. T. Schaub, R. Lambiotte, and M. Barahona, Encoding dynamics for multiscale community detection: Markov time sweeping for the map equation, Physical Review E **86**, 026112 (2012).
[72] M. Kheirkhahzadeh, A. Lancichinetti, and M. Rosvall, Efficient community detection of network flows for varying Markov times and bipartite networks, Physical Review E **93**, 032309 (2016).
[73] P. Pons and M. Latapy, Computing communities in large networks using random walks, Journal of Graph Algorithms and Applications **10**, 191 (2006).
[74] M. Coscia, G. Rossetti, F. Giannotti, and D. Pedreschi, DEMON: a local-first discovery method for overlapping communities, in *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2012) pp. 615–623.
[75] G. Rossetti, ANGEL: efficient, and effective, node-centric community discovery in static and dynamic networks, Applied Network Science **5**, 26 (2020).
[76] T. P. Peixoto, The graph-tool python library, figshare (2014).
[77] J. Yang and J. Leskovec, Overlapping community detection at scale: A nonnegative matrix factorization approach, in *Proceedings of the Sixth ACM International*

Conference on Web Search and Data Mining (2013) pp. 587–596.

[78] A. Epasto, S. Lattanzi, and R. Paes Leme, Ego-splitting framework: From non-overlapping to overlapping clusters, in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2017) pp. 145–154.

[79] A. Lancichinetti, S. Fortunato, and J. Kertész, Detecting the overlapping and hierarchical community structure in complex networks, New Journal of Physics **11**, 033015 (2009).

[80] L. M. Collins and C. W. Dent, Omega: A general formulation of the Rand index of cluster recovery suitable for non-disjoint solutions, Multivariate Behavioral Research **23**, 231 (1988).

[81] A. F. McDaid, D. Greene, and N. Hurley, Normalized mutual information to evaluate overlapping community finding algorithms (2011), arXiv:1110.2515.

[82] A. Lancichinetti and S. Fortunato, Limits of modularity maximization in community detection, Physical Review E **84**, 066122 (2011).

[83] T. P. Peixoto, The Netzschleuder network catalogue and repository (2020), https://networks.skewed.de.

[84] J. McAuley and J. Leskovec, Learning to discover social circles in ego networks, in *Advances in Neural Information Processing Systems 25* (2012) pp. 539–547.

[85] M. Girvan and M. E. J. Newman, Community structure in social and biological networks, Proceedings of the National Academy of Sciences **99**, 7821 (2002).

[86] V. Krebs, Books about US politics, unpublished network of Amazon.com co-purchases, http://www.orgnet.com/.

[87] M. E. J. Newman, Modularity and community structure in networks, Proceedings of the National Academy of Sciences **103**, 8577 (2006).

[88] L. A. Adamic and N. Glance, The political blogosphere and the 2004 U.S. election: divided they blog, in *Proceedings of the 3rd International Workshop on Link Discovery* (2005) pp. 36–43.

[89] J. Stehlé, N. Voirin, A. Barrat, C. Cattuto, L. Isella, J.-F. Pinton, M. Quaggiotto, W. Van den Broeck, C. Régis, B. Lina, and P. Vanhems, High-resolution measurements of face-to-face contact patterns in a primary school, PLoS ONE **6**, e23176 (2011).

[90] R. J. G. B. Campello, D. Moulavi, and J. Sander, Density-based clustering based on hierarchical density estimates, in *Advances in Knowledge Discovery and Data Mining (PAKDD 2013)*, Lecture Notes in Computer Science, Vol. 7819 (2013) pp. 160–172.